



Tema 3 - TCP

Mgtr. Lic. Barrios Walter G.

Necesidad servicio confiable

- Las redes de comunicación no garantizan la entrega de paquetes.
- Los paquetes pueden perderse, retrasarse, duplicarse o llegar desordenados.
- La sobrecarga y los cambios dinámicos de ruta pueden aumentar estos problemas.
- Las aplicaciones que transfieren grandes cantidades de datos necesitan comunicación confiable.

Necesidad servicio confiable

- Sin un protocolo confiable, cada aplicación debería implementar su propia detección y recuperación de errores.
- Esto aumenta la complejidad y dificultad del desarrollo.
- TCP proporciona un servicio de transferencia confiable y general, reutilizable por diferentes aplicaciones.

TCP centraliza la confiabilidad de la comunicación para que las aplicaciones no tengan que implementar por separado mecanismos de detección y recuperación de errores.

Propiedades

Lo que TCP ofrece a las aplicaciones puede caracterizarse por estos 5 puntos

- Orientado a flujo (Stream)
- Conexión de circuito virtual
- Transferencia con buffer
- Flujo no estructurado
- Comunicación full duplex

Orientado a flujo

- TCP trata los datos como un flujo continuo de octetos (bytes).
- La aplicación receptora recibe los datos en el mismo orden en que fueron enviados.
- TCP garantiza la entrega ordenada y confiable del flujo.
- La aplicación no necesita preocuparse por cómo TCP divide los datos en segmentos.

Conexión de circuito virtual

- Antes de transmitir datos, TCP establece una conexión entre emisor y receptor.
- Ambos extremos verifican que la comunicación esté autorizada y lista.
- Una vez establecida, comienza la transferencia de datos.
- TCP supervisa la comunicación y detecta posibles fallos durante la transferencia.
- Si ocurre un problema en la red, TCP informa a las aplicaciones involucradas.
- Se denomina circuito virtual porque funciona como una conexión dedicada, aunque los datos viajen en paquetes.

Transferencia con buffer

- La aplicación puede enviar datos en bloques de cualquier tamaño, incluso un octeto.
- TCP almacena los datos en un buffer y los divide en paquetes según sea conveniente.
- Normalmente, TCP acumula datos para formar paquetes más grandes y eficientes, reduciendo el tráfico de red.
- Si la aplicación genera bloques muy grandes, TCP puede dividirlos en varios paquetes.

Transferencia con buffer

- El receptor obtiene los datos en el mismo orden en que fueron enviados.
- El mecanismo PUSH permite solicitar que los datos almacenados se transmitan sin esperar a llenar el buffer.
- PUSH no conserva los límites de los bloques originales de la aplicación.

Flujo no estructurado

- TCP transmite los datos como un flujo continuo de octetos.
- No identifica automáticamente el tipo de información ni separa los datos en registros.
- Las aplicaciones deben definir previamente un formato para interpretar los datos y establecer sus propios límites.

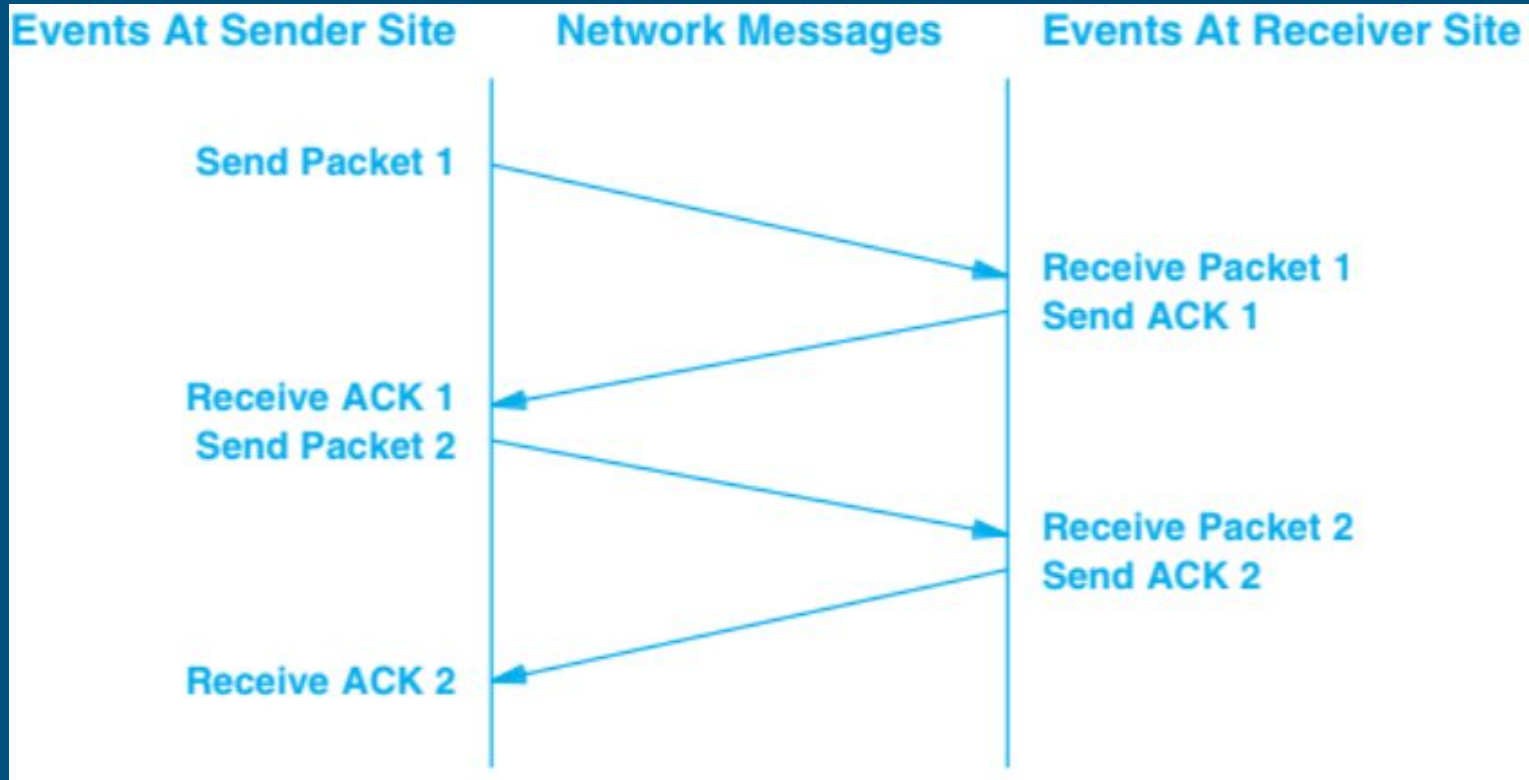
Comunicación Full Duplex

- TCP permite la transferencia simultánea de datos en ambas direcciones.
- Cada dirección funciona como un flujo de datos independiente.
- Es posible cerrar la comunicación en una dirección mientras la otra continúa (half duplex).
- TCP puede enviar información de control junto con los datos en sentido contrario (piggybacking), reduciendo el tráfico de red.

Confiabilidad: confirmaciones y retransmisión

- TCP garantiza la entrega de datos sin pérdidas ni duplicaciones.
- Utiliza el mecanismo PAR (Positive Acknowledgement with Retransmission).
- El receptor envía una confirmación (ACK) cuando recibe correctamente los datos.
- El emisor inicia un temporizador al enviar un paquete.
- Si no recibe el ACK a tiempo, retransmite el paquete

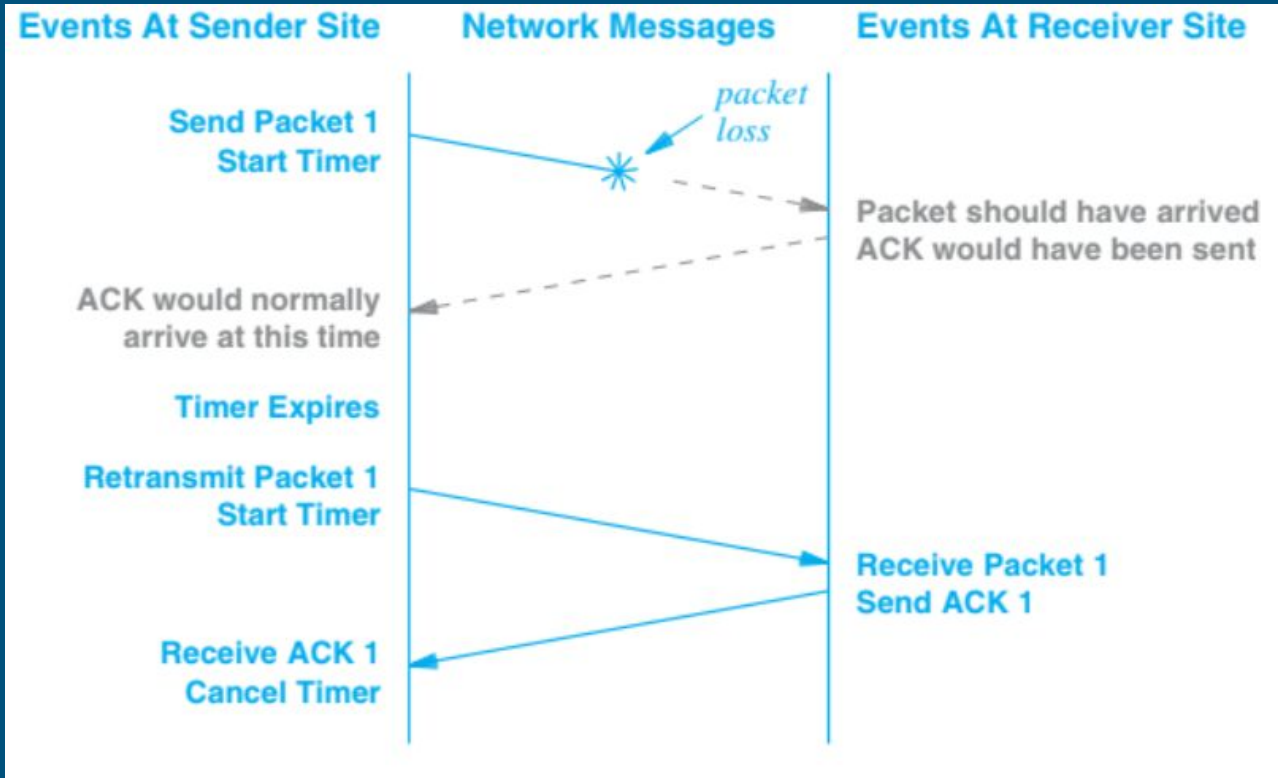
Transmisión simple



Transmisión simple

- Es el mecanismo de retransmisión más simple.
- El emisor envía un paquete y espera su confirmación (ACK).
- Solo después de recibir el ACK, envía el siguiente paquete.
- Es sencillo y confiable, pero poco eficiente, porque no permite enviar varios paquetes simultáneamente.

Transmisión simple



Transmisión simple

- Si un paquete se pierde, el emisor espera hasta que expire el temporizador y lo retransmite.
- El emisor conserva los datos no confirmados para poder retransmitirlos.
- Un paquete también puede llegar duplicado, por ejemplo, debido a retrasos en la red.
- TCP utiliza números de secuencia para identificar cada paquete.
- Las confirmaciones (ACK) incluyen el número de secuencia recibido, permitiendo asociar cada ACK con su paquete correspondiente.
- Así, TCP puede detectar pérdidas, retransmisiones y duplicados.

Paradigma ventana deslizante

Initial window



(a)

Window slides →



(b)

Ventana deslizante

- Permite enviar varios paquetes sin esperar un ACK por cada uno.
- Utiliza una ventana de tamaño fijo que determina cuántos paquetes pueden enviarse.
- Mejora el throughput y aprovecha mejor la capacidad de la red.
- Evita la inactividad que ocurre con Send-and-Wait, especialmente en redes con alta latencia.

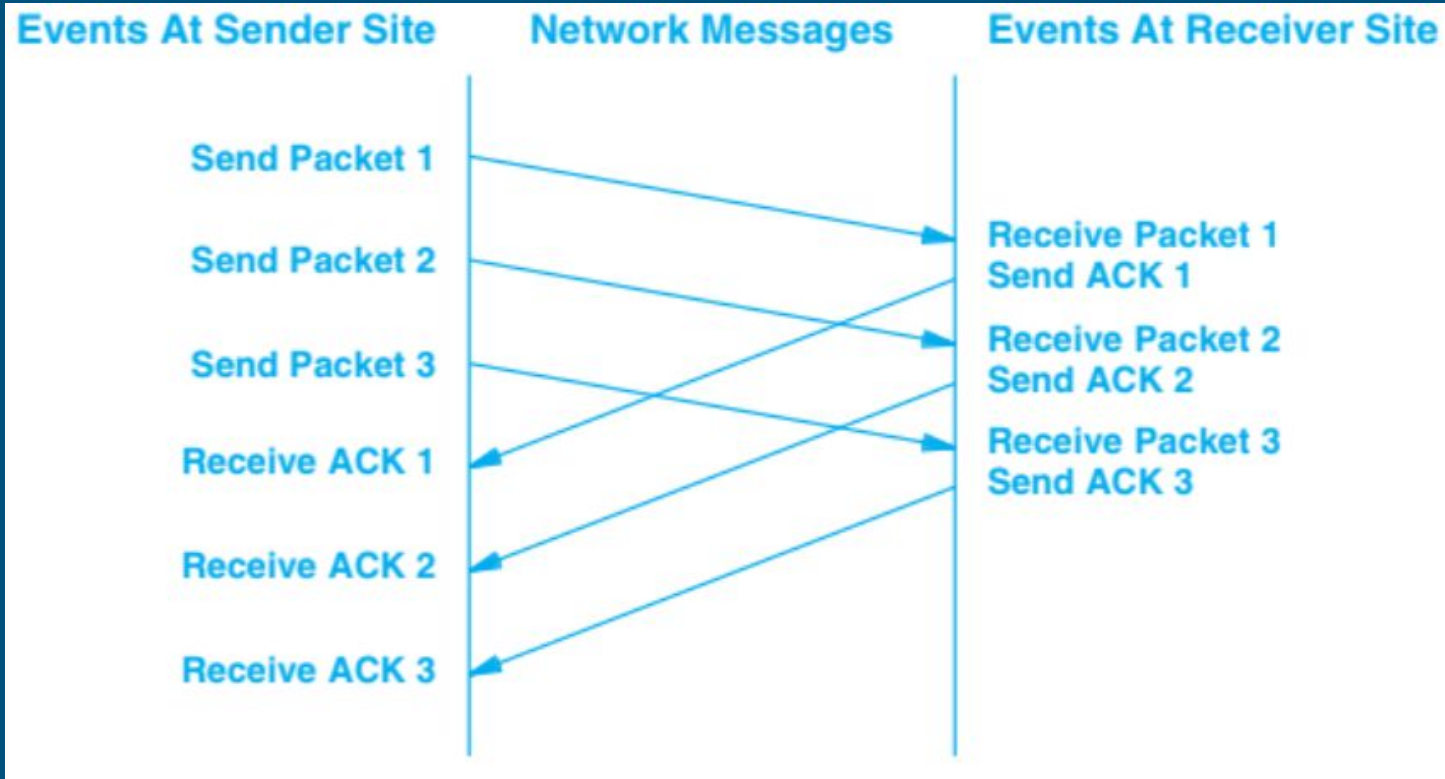
La ventana deslizante permite transmitir varios paquetes en paralelo antes de recibir sus confirmaciones, aumentando la eficiencia de la comunicación.

Ventana deslizante

- Mantiene la red ocupada continuamente, logrando mayor throughput.
- Lleva el control de los paquetes confirmados y pendientes.
- Cada paquete sin confirmar tiene un temporizador; si expira, se retransmite.
- La ventana avanza a medida que recibe ACKs.
- Divide los paquetes en tres grupos:
 -  Confirmados: ya recibidos correctamente.
 -  En transmisión: enviados pero aún sin confirmar.
 -  Pendientes: todavía no enviados.

La ventana deslizante mejora el rendimiento al mantener varios paquetes en tránsito y retransmitir únicamente los que no son confirmados.

Ventana deslizante



Protocolo de Control de Transmisión (TCP)

- TCP proporciona un servicio de flujo de datos confiable.
- Utiliza mecanismos como la ventana deslizante para controlar la transmisión.
- TCP es un protocolo de comunicación, no un software.
- El protocolo TCP define las reglas de comunicación.
- El software TCP implementa esas reglas.
- TCP es fundamental en Internet, de ahí el nombre TCP/IP.

TCP define cómo se realiza una comunicación confiable; el software TCP implementa esas reglas.

¿Qué proporciona TCP?

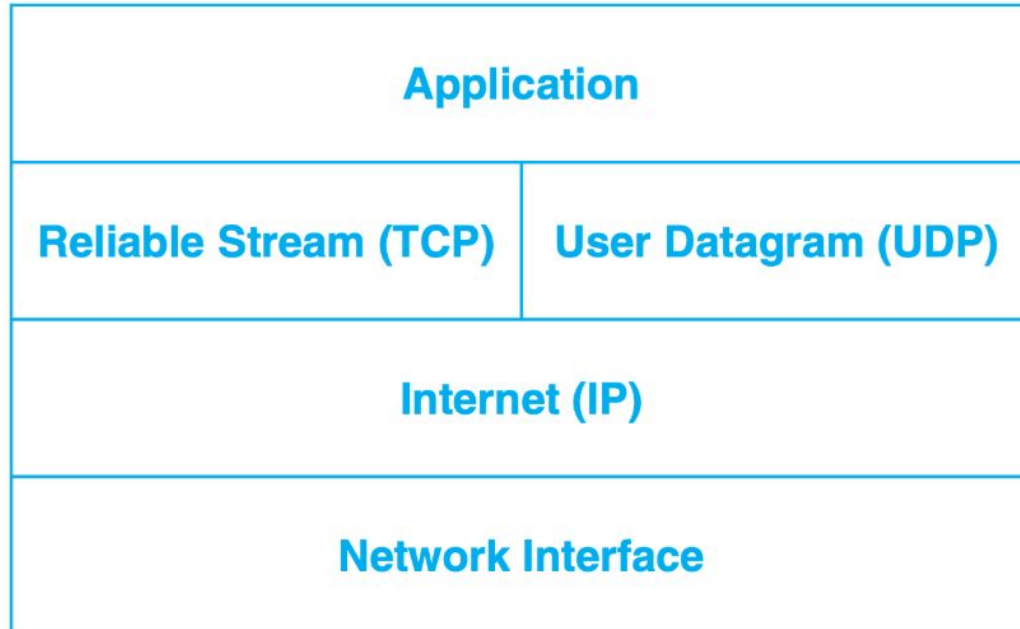
- Define el formato de los datos y ACKs para lograr una transmisión confiable.
- Garantiza la correcta entrega y maneja pérdidas, duplicados y errores.
- Permite identificar múltiples destinos dentro de una misma computadora.
- Define cómo iniciar y finalizar una conexión TCP.
- No establece una interfaz específica entre las aplicaciones y TCP, permitiendo adaptarse a distintos sistemas operativos.
- Puede funcionar sobre diferentes tipos de redes, incluso con alta latencia, pérdidas o congestión.
- TCP proporciona una comunicación confiable y adaptable, independientemente de la red subyacente y del sistema operativo utilizado.

Capas, puertos, conexiones y puntos finales

- TCP pertenece a la capa de transporte, por encima de IP.
- Permite que varias aplicaciones se comuniquen simultáneamente en una misma computadora.
- Utiliza puertos para identificar y dirigir los datos hacia la aplicación correspondiente.
- Demultiplexa el tráfico entrante, entregando cada conexión a la aplicación correcta.
- TCP cumple una función similar a UDP, ya que ambos son protocolos de la capa de transporte.
- TCP permite que múltiples aplicaciones compartan la red simultáneamente, identificándose mediante puertos y conexiones.

Capas, puertos, conexiones y puntos finales

Conceptual Layering



Puntos finales (Endpoints) de TCP

- Un punto final está formado por IP + puerto TCP.
- Una conexión TCP se identifica mediante dos puntos finales: origen y destino.
- Un mismo puerto TCP puede ser utilizado por varias conexiones simultáneas.
- TCP distingue cada conexión utilizando ambos puntos finales, por lo que no existe ambigüedad.
- Esto permite que una aplicación atienda múltiples conexiones al mismo tiempo utilizando un único puerto local.

Puntos finales (Endpoints) de TCP

Ejemplo:

(128.10.2.3, 53) puede participar en varias conexiones:

- (128.9.0.32, 1184) $\leftrightarrow$ (128.10.2.3, 53)
- (128.2.254.139, 1184) $\leftrightarrow$ (128.10.2.3, 53)

TCP identifica las conexiones mediante el par IP + puerto de ambos extremos, permitiendo que un mismo puerto atienda múltiples conexiones simultáneamente.

Aperturas pasivas y activas

- TCP requiere que ambos extremos acepten la conexión antes de intercambiar datos.
- Apertura pasiva: un programa queda preparado para aceptar conexiones entrantes en un puerto determinado.
- Apertura activa: el otro programa solicita iniciar la conexión TCP.
- Los módulos TCP de ambos extremos negocian y verifican la conexión.
- Una vez establecida, comienza la transferencia confiable de datos.
- La conexión TCP se establece cuando un extremo espera conexiones (apertura pasiva) y el otro solicita conectarse (apertura activa)

Segmentos, flujos y números de secuencia

- TCP divide el flujo de datos en segmentos para transmitirlos.
- Cada segmento normalmente viaja dentro de un datagrama IP.
- Utiliza una ventana deslizante para mejorar el rendimiento y controlar el flujo.
- TCP puede enviar varios segmentos antes de recibir ACKs.

Segmentos, flujos y números de secuencia

La ventana TCP tiene tres límites principales:

- Octetos enviados y confirmados.
- Octetos enviados pero aún sin confirmar.
- Octetos que todavía pueden enviarse.

Segmentos, flujos y números de secuencia

TCP es Full Duplex, por lo que puede transmitir y recibir datos simultáneamente.

- Cada dirección de comunicación funciona de manera independiente.
- Por cada conexión, TCP mantiene dos ventanas:
 -  Una para controlar los datos enviados.
 -  Otra para controlar los datos recibidos.
- Ambas ventanas se deslizan de forma independiente según avanza la comunicación.

Tamaño de ventana variable y control de flujo

- TCP permite que el tamaño de la ventana cambie dinámicamente.
- El receptor anuncia en cada ACK cuántos octetos adicionales puede aceptar.
- Si el buffer se llena, el receptor reduce la ventana para limitar la transmisión.
- Una ventana de 0 detiene temporalmente el envío.
- Cuando tiene espacio, el receptor aumenta la ventana y se reanuda la transmisión.

Tamaño de ventana variable y control de flujo

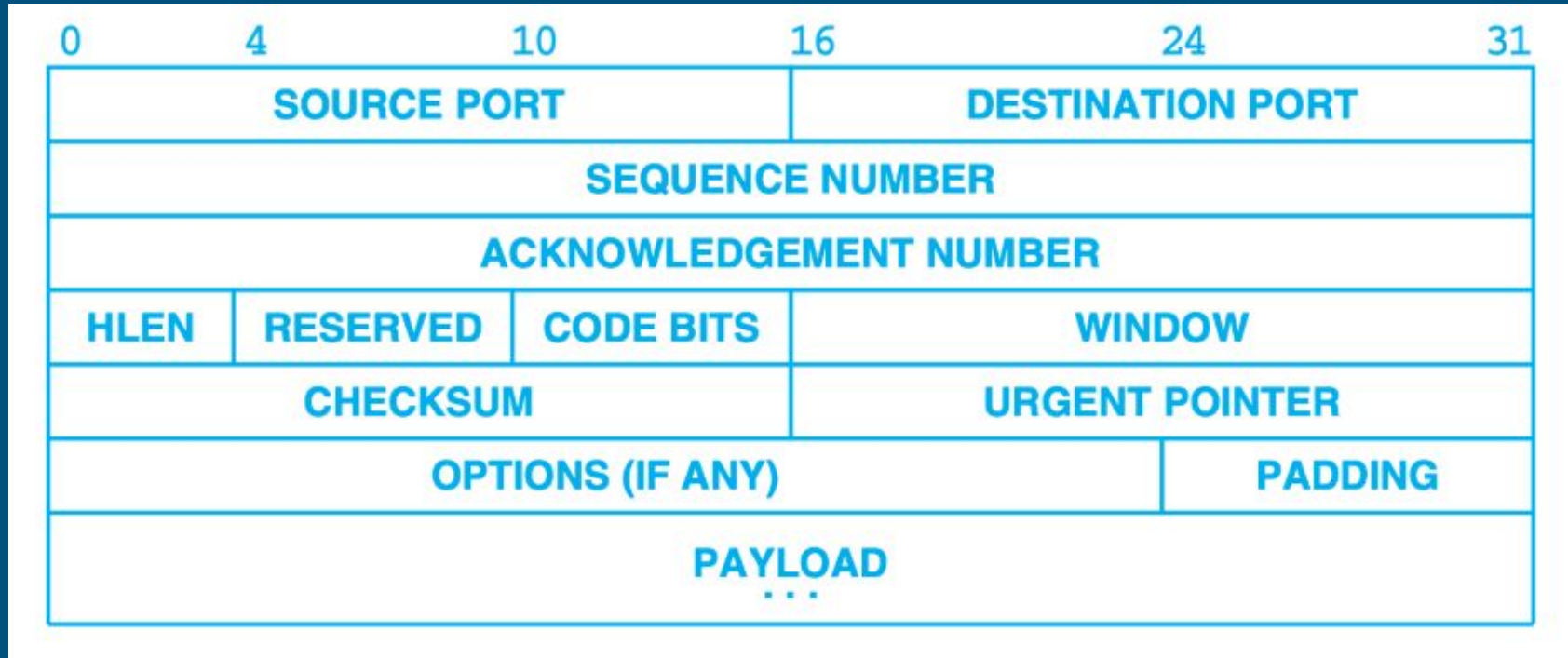
- Esto permite realizar control de flujo extremo a extremo, adaptándose a las capacidades del receptor.
- El control de flujo evita desbordar al receptor.
- El control de congestión busca evitar la sobrecarga de la red y los routers.

TCP ajusta dinámicamente su ventana para controlar el flujo de datos y evitar que el receptor se sobrecargue.

Formato del segmento TCP

- La unidad de transferencia de TCP se denomina segmento.
- Los segmentos permiten:
 - Establecer y cerrar conexiones.
 - Transferir datos.
 - Enviar ACKs.
 - Anunciar el tamaño de ventana.
- TCP permite, combinar datos y confirmaciones en un mismo segmento.
- Un segmento TCP tiene dos partes principales:
 - Encabezado: contiene información de control y metadatos.
 - Payload (carga útil): contiene los datos de la aplicación.

Formato del segmento TCP



Formato del segmento TCP

- **Source Port / Destination Port:** identifican las aplicaciones de origen y destino.
- **Sequence Number:** indica la posición de los datos dentro del flujo enviado.
- **Acknowledgement Number:** indica el próximo octeto que se espera recibir.
- **HLEN:** indica el tamaño del encabezado TCP.
- **Code Bits:** indican la función del segmento, como establecer, transferir o cerrar una conexión.
- **Window:** indica cuántos datos puede recibir el receptor.
- **Options:** contiene opciones adicionales y puede aumentar el tamaño del encabezado.
- **Reserved:** bits reservados para usos futuros.

Opciones de TCP

- El encabezado TCP puede incluir cero o más opciones.
- Cada opción contiene:
 - Tipo: identifica la opción.
 - Longitud: indica su tamaño.
- Las opciones permiten agregar información adicional al encabezado TCP.
- El encabezado debe tener un tamaño múltiplo de 32 bits.
- Si es necesario, se agrega padding (relleno) para completar el tamaño.

Las opciones permiten ampliar la información del encabezado TCP manteniendo su tamaño alineado a múltiplos de 32 bits.

Opción MSS (Maximum Segment Size)

- El MSS define el tamaño máximo de datos que puede transportar un segmento TCP.
- Permite que emisor y receptor se adapten a sus capacidades de buffer.
- Un MSS adecuado ayuda a maximizar el rendimiento y evitar fragmentación IP.
- Segmentos muy pequeños → mucho overhead de encabezados y menor eficiencia.

Opción MSS (Maximum Segment Size)

- Segmentos muy grandes → pueden provocar fragmentación IP y aumentar la posibilidad de pérdida.
- Idealmente, el MSS debe ajustarse al MTU de la ruta para evitar fragmentación.
- El tamaño óptimo puede variar si cambia la ruta de red o existen diferentes encabezados IP.